Agent Runtime / Harness 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉沙箱、编排引擎、Guardrails 等概念，可以直接翻主报告。

> 🎯 **读完这篇你能**：评估一个 Agent 项目的安全需求，选择合适级别的沙箱方案（API 级 / Firecracker / gVisor）和编排引擎（图编排 / 可视化编排），并设计基本的 Guardrails 和可观测性策略。

一、Agent Runtime 是什么?

让 AI Agent **跑起来的底层设施**。Agent 不只是"调一次 LLM 得到一个回答"——它是持续运行的进程，需要执行代码、访问文件、调用 API、记住状态、在出错时恢复。这些"让 Agent 稳定运行"的能力就是 Runtime 提供的。

打一个直觉比方：如果 Agent 是一辆车，Model（LLM）是发动机，Framework 是车身，**Runtime/Harness 就是刹车、方向盘、安全带和仪表盘**——没有它，发动机再强也上不了路。

二、核心概念

沙箱（Sandbox）

一个**隔离的执行环境**，Agent 在其中运行代码但无法影响外部系统。就像把 Agent 关在一个"沙盒"里——它可以在里面随便玩，但破坏不了外面的东西。

四种方案：

- **API 级沙箱**（OpenAI）：限制 Agent 只能调用白名单 API，不准碰文件系统
- **Firecracker**（AWS）：每个 Agent 会话跑在一个微型虚拟机里（~125ms 启动）（Agache et al., AWS, 2020）
- **gVisor**（Google）：在应用和内核之间加一层"拦网"
- **Tank OS**（Red Hat）：把整个 Agent 环境打包成不可变的 Linux 镜像

编排引擎（Orchestration Engine）

管理多个 Agent 的协作流程。一个复杂任务通常需要多个 Agent 分工——一个查资料、一个写代码、一个做测试。编排引擎负责"谁先做、谁后做、做错了怎么恢复"。

两种主流方式：

- **图编排**（LangGraph）：用有向图定义步骤和分支。**类比**：像画流程图——你先画好 A→B→C 的路线图，代码自动按图执行。
- **可视化编排**（Dify）：拖拽节点，用箭头连线。**类比**：像用 PPT 画架构图——拖框框、画箭头，不写代码也能定义流程。

选择哪一种取决于团队——开发团队偏爱图编排（灵活可控），业务团队偏爱可视化编排（无需编程）。

Guardrails（护栏）

Agent 的**安全过滤层**。像一个门卫，检查 Agent 的输入和输出有没有问题（NVIDIA）。

三种护栏：

- **话题护栏**：不准 Agent 讨论某些话题
- **内容护栏**：不准输出有害内容
- **越狱护栏**：检测并阻止 Prompt Injection

MCP（Model Context Protocol，模型上下文协议）Gateway

MCP 协议的流量入口（Anthropic, 2024）。就像 Web 应用的 API Gateway——所有 Agent 到外部工具的连接都经过 Gateway，在这里做认证、速率限制、审计日志。

可观测性（Observability）

Agent 的**运行监控系统**。传统应用监控的是"这个 API 返回了 200 还是 500"，Agent 监控的是"LLM 在这一步想了什么、调了什么工具、状态怎么变的"。OpenTelemetry 正在为 Agent 定义标准追踪格式。

类比：就像飞机的黑匣子——不仅记录"飞机在飞"（传统监控），还记录"飞行员在想什么、按了什么按钮、仪表盘读数怎么变的"（Agent 的行为轨迹）。没有可观测性的 Agent 像个黑箱——出了问题你完全不知道它做了什么决策。

Durable Execution（持久化执行）

Agent 在任何步骤中断后都能恢复到中断前的状态。想象你让 Agent 做一个三天的研究任务，它在第 2.8 天崩溃了——没有 Durable Execution 就得重来。Temporal 是这个领域的代表方案（Temporal Technologies）。

类比：就像玩一个不能存档的 RPG 游戏——你打了 28 个小时，电脑突然死机了，得从头开始。Durable Execution 就是游戏的自动存档功能——每次关键节点自动保存，任何时候崩溃都能从最近存档恢复。

三、一个简单直觉

```
Agent Runtime 的进化 = 从"马路上随便跑"到"有交通规则"
2023（萌芽期）：没有规则，谁都能跑，出了事没人管
2024（框架期）：有了车道线（编排），但还没有交警和红绿灯
2025（基建期）：沙箱、护栏、监控全上——像修好了高速公路
2026（Harness 范式）：不是"更好的路"，而是"操作系统"——统一管理所有车的运行
```

Agent Runtime 在 2026 年还是一个非常年轻的领域——OpenAI 刚把 Harness 定义出来（OpenAI, 2025），Martin Fowler 刚把它列为正式工程实践。未来很可能会像 DevOps 一样，成为一个专门的工种。

参考来源

- **OpenAI Agents SDK** (OpenAI, 2025) — OpenAI 官方 Agent 运行时框架，定义了 Agent Harness 的范式，含 Guardrails、Handoff、Tracing — https://openai.github.io/openai-agents-python/
- **Temporal** (Temporal Technologies) — 持久化执行引擎代表，支持工作流在任何步骤中断后恢复到中断前状态 — https://temporal.io/
- **Firecracker: Lightweight Virtualization for Serverless Applications** (Agache et al., AWS, 2020) — AWS 开源的微型虚拟机，每个 Agent 会话可跑在独立 microVM 中，~125ms 启动 — https://firecracker-microvm.github.io/
- **gVisor** (Google) — 应用级内核沙箱，在应用和 Linux 内核之间加一层拦截层，为容器提供额外隔离 — https://github.com/google/gvisor
- **Model Context Protocol Specification** (Anthropic, 2024) — 统一 Agent 与外部工具/数据源连接方式，MCP Gateway 提供认证、限流和审计能力 — https://modelcontextprotocol.io/
- **NVIDIA NeMo Guardrails** (NVIDIA) — 开源 Guardrails 框架，支持话题护栏、内容护栏和越狱检测，可与任何 LLM 集成 — https://github.com/NVIDIA/NeMo-Guardrails